Portfolio

About Me

저를 소개합니다



박민수 Park Minsoo

010-9705-7535

palstn97@naver.com

1997.06.24

서비스와 사용자를 연결하는 개발자

복잡한 요구사항을 명확하고 안정적인 사용자 경험으로 연결하는 것을 강점으로 합니다.
기능을 단순히 구현하는 것을 넘어, 사용자의 흐름을 자연스럽게 만들기 위한 설계, 예외 상황에서도 끊김 없는 경험을 위한 문제 분석과 서비스 성능을 높이기 위해 언제나 노력하고 있습니다.

Timeline

걸어온 길

2025.01 ~ ing

SSAFY

교육생

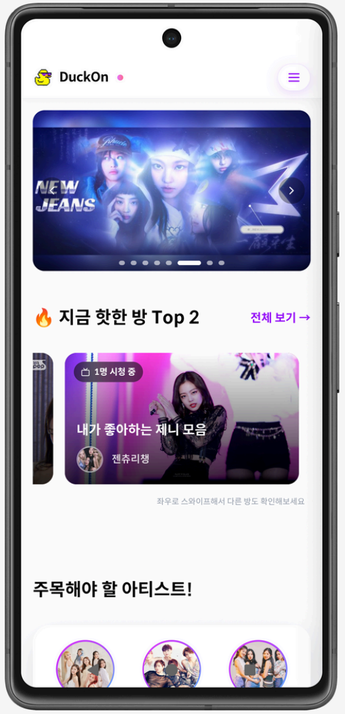
2019.03 ~ 2025.02

광운대학교 전자재료공학과

Project 01

진행 프로젝트 소개





DuckOn

프로젝트

DuckOn은 다양한 아티스트의 전세계 팬들이 실시간으로 함께 영상을 보고 채팅으로 소통할 수 있는 플랫폼입니다.

개요

- ✔ 기간: 2025.07 ~ ing
- ✔ 역할: 프론트엔드
- ✔ 팀 구성: 5인 팀(백엔드 3명, 프론트엔드 2명)
- ✔ 목표: 팬들이 실시간으로 함께 시청하고 대화할 수 있는 공간 제공

핵심 기능

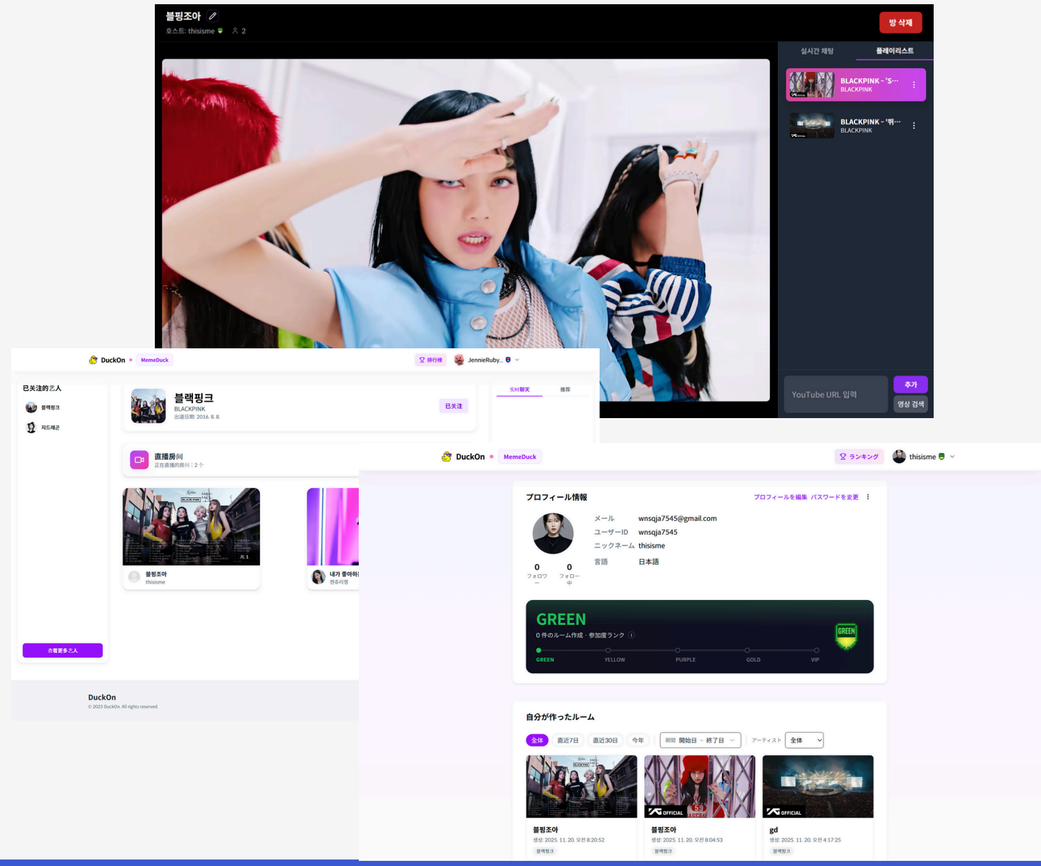
- ✔ 실시간 영상 동기화
- ✔ 실시간 채팅
- ✔ 각 아티스트별 팬 전용 공간
- ✔ 글로벌 팬들을 위한 실시간 번역 기능
- ✔ MemeDuck과의 연동을 통한 채팅 GIF 전송

DuckOn

Project 01

담당 역할

- 1000명 이상의 사용자간의 실시간 영상 동기화
  - YouTube Player API를 활용한 드리프트 보정 알고리즘 개발
  - 네트워크 지연, 브라우저 성능 차이에 따른 재생 오차 최소화
- 실시간 채팅 구현 및 UI, 성능 최적화
  - 채팅 폭주 상황에서 렌더링 병목 해결
  - 자동 스크롤 조건 처리, ‘새 채팅 도착’ 배지 등 사용자 경험 개선
  - 도배 방지 알고리즘 구현
  - 차단 및 강제 퇴장 UI 및 상태 관리 구현
- Capacitor 기반 모바일 앱 패키징
  - 기존 웹코드를 WebView 기반 모바일 앱으로 전환
- 다국어 지원 서비스
  - i18n 개반 UI 텍스트 분리
- YouTube Data API v3를 활용한 방 생성 및 플레이리스트 설정



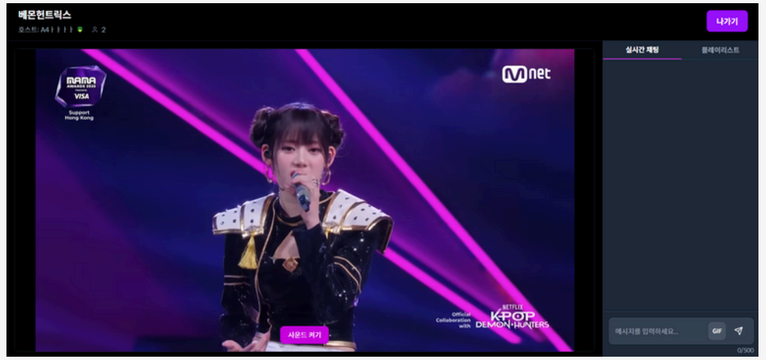
DuckOn

Project 01

트러블슈팅 & 해결

- 실시간 영상 동기화 및 끊김 현상
  - 방장과 참여자 간 영상 재생 시점이 계속 어긋나는 현상 발생
  - 하트비트 수신 시 영상이 2 ~ 3초 간격으로 끊겨 보이는 현상 발생
  - 해결 방안
    - currentTime과 lastUpdated를 기반으로 예상 재생 시각 계산
      - expectedTime = host.currentTime + (now - lastUpdated) / 1000
    - 로컬 플레이어 시간과의 차이를 수치로 계산하여 드리프트 기준을 정의
      - DRIFT_TOLERANCE_HARD(2.5초) 이상일 때만 하드 보정
      - DRIFT_TOLERANCE_SOFT(0.2~0.4초) 이상일 때는 일시적으로 재생 속도 1.25배, 0.75배로 소프트 보정
    - 웹/앱 환경에 따라 드리프트 임계값과 하트비트 주기를 분리
      - 모바일에서는 네트워크 및 성능 특성을 고려해 더 촘촘한 보정 주기와 낮은 소프트 임계값 적용
    - 이를 통해 필요할 때만 보정하는 자연스러운 동기화 개선



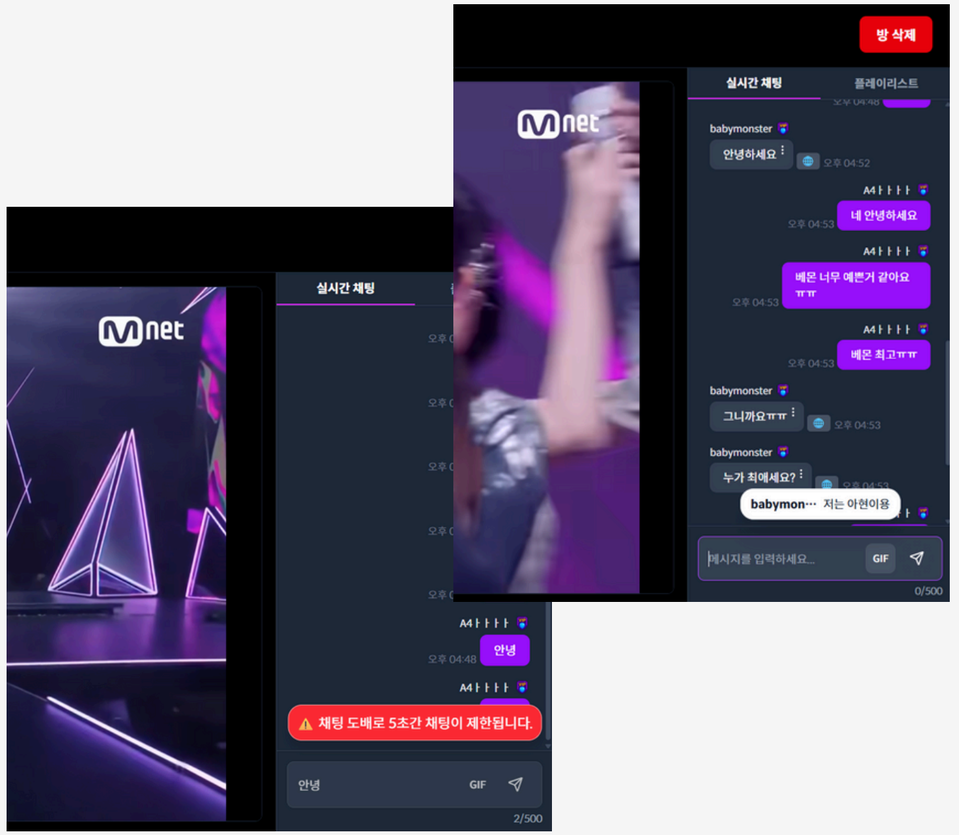


DuckOn

Project 01

트러블슈팅 & 해결

- 실시간 채팅 UI 및 도배 방지
  - 사용자 스크롤 위치와 상관없이 새 채팅이 오면 항상 최근 채팅 내역으로 이동
  - 과거 채팅을 읽는 중에도 맨 아래로 내려가는 불편함
  - 해결 방안
    - 현재 스크롤 위치를 숫자로 판단하는 로직 추가를 통해 '가장 아래에 위치' 할 때만 자동 스크롤
    - 사용자가 과거의 채팅을 보고 있다면 자동 스크롤 발생하지 않도록 차단
    - 사용자들의 편의성을 위해 '새 채팅' 알림 배지 구현 후 배지 클릭 시 자동 이동
    - 채팅 미리보기 UX 적용
    - 내가 보낸 채팅은 항상 맨 아래로 이동을 통해 자연스러운 UX 적용
  - 수십 개의 채팅을 보내는 도배 사용자로 WebSocket 이벤트 폭증
  - 해결 방안
    - 5초간 메시지 전송 타임스탬프 배열 관리하여 반복 전송 패턴을 숫자로 감지
    - 도배 시 5초간 채팅 전송 전면 차단
    - 차단이 끝날 때 다시 정상적으로 채팅을 보낼 수 있도록 안정 상태 유지
    - 도배 방지 배지를 띄워 사용자가 명확하게 확인할 수 있도록 안내

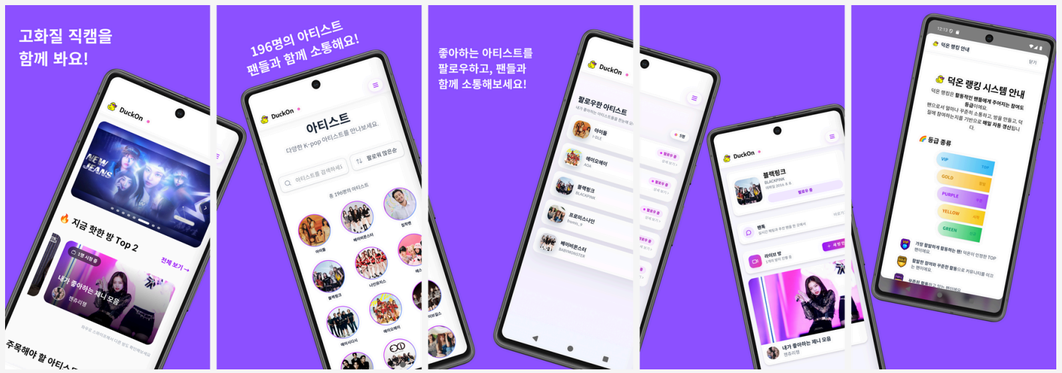


DuckOn

Project 01

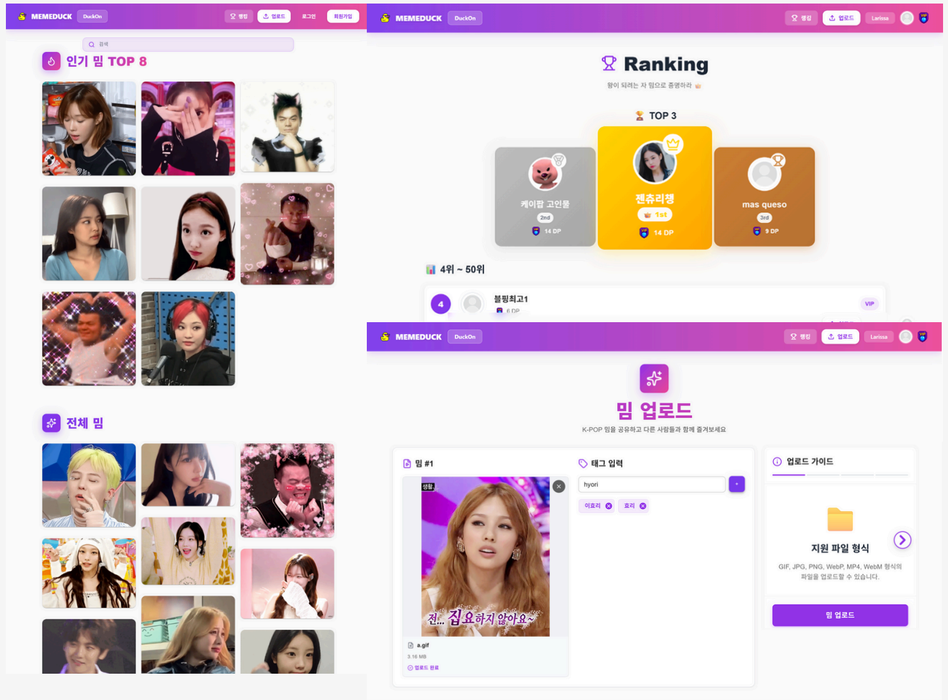
트러블슈팅 & 해결

- WebView 환경에서 WebSocket 미작동 및 어색한 UI
  - Android WebView 환경에서 WebSocket 연결 실패
  - 방 입장 시 영상 동기화, 채팅 수신 불가
  - 웹 기준으로 제작된 UI가 모바일 WebView에서 비율 깨짐, 반응형 미작동 등의 오류 발생
  - 해결 방안
    - WS 주소절대경로로 고정
    - 서버 CORS에 capacitor 추가 허용
    - 재연결 및 Heartbeat 설정 강화
      - 그 결과 앱에서도 채팅, 영상 동기화 정상 작동
    - 웹과 앱의 UI 완전 분리 전략 적용
    - isNativeApp 조건으로 앱 전용 레이아웃 제공
    - 반응형 재구축, 탭 구조 재정의
      - 그 결과 앱 UI가 Web과 완전히 독립된 형태로 동작
      - 가독성, 터치 편의성, 정렬 문제 모두 해결



Project 02

진행 프로젝트 소개



MemeDuck

프로젝트

MemeDuck은 밈을 직접 생성하고 공유할 수 있는 아카이빙 플랫폼으로 기존의 서비스였던 DuckOn에서 실시간 채팅 중 생성한 밈을 직접 사용 가능하도록 하였습니다.

개요

- ✔ 기간: 2025.10 ~ ing
- ✔ 역할: 프론트엔드
- ✔ 팀 구성: 5인 팀(백엔드 3명, 프론트엔드 2명)
- ✔ 목표: 팬들이 아티스트와 관련한 밈을 직접 생성하고 사용 가능한 환경 구축

핵심 기능

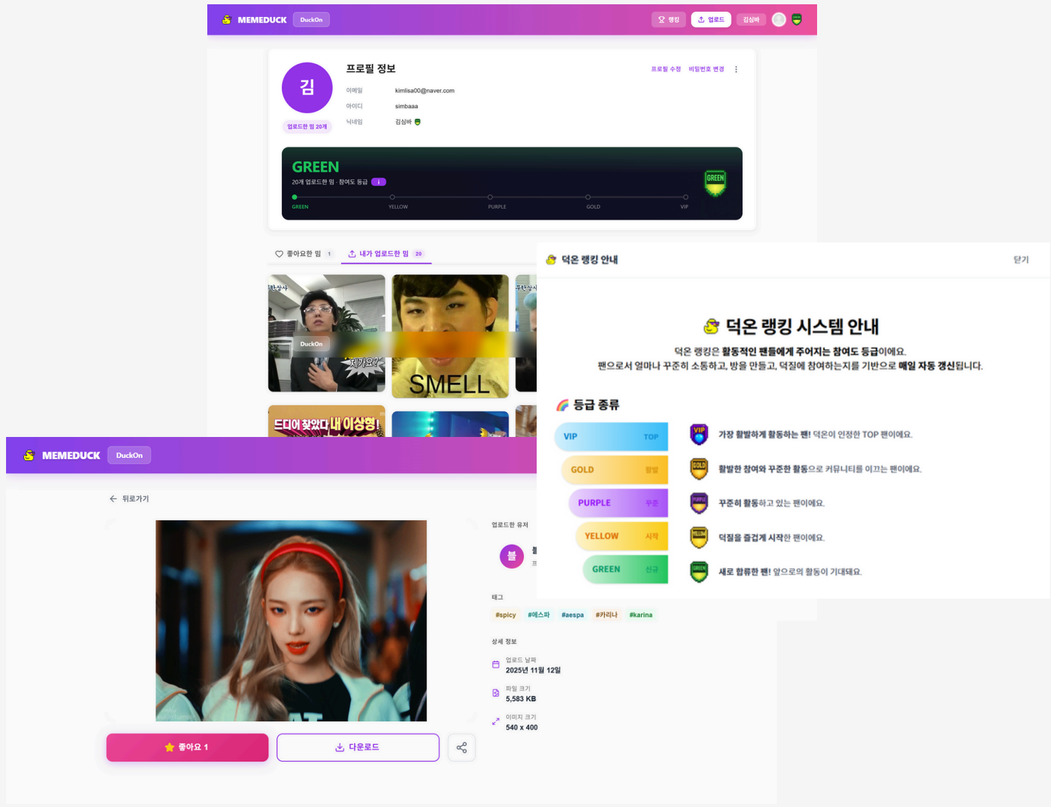
- ✔ 밈 생성 및 공유
- ✔ DuckOn과의 연동 및 활용성 강화
- ✔ 랭킹 시스템을 통한 차별화
- ✔ 사용량 실시간 집계

MemeDuck & DuckOn

Project 02

담당 역할

- GIF 전송을 위한 메시지 구조 처리
  - STOMP 기반 채팅 메시지 구조 확장하여 GIF URL/이미지 전송을 안정적으로 처리
  - 이미지 채팅과 텍스트 채팅의 파싱 분리
- 밈 상세 페이지 및 리스트 UI 개발
  - 밈(GIF) 리스트 페이지 렌더링 최적화 → Skeleton UI 적용
  - 상세 페이지에서 태그, 다운로드 수, 좋아요 수 등 메타 정보 시각화
- 밈 검색 모달 UI/UX 개발 및 DuckOn 내 연동
  - 실시간 검색 기반 모달 UI 개발
  - 검색된 GIF를 클릭 한 번으로 DuckOn 채팅으로 전송 가능
- DuckOn 채팅 이미지 렌더링 및 스크롤 안정화 로직 구축
- 랭킹 시스템
  - 유저 활동 기반 실시간 랭킹 UI 구현

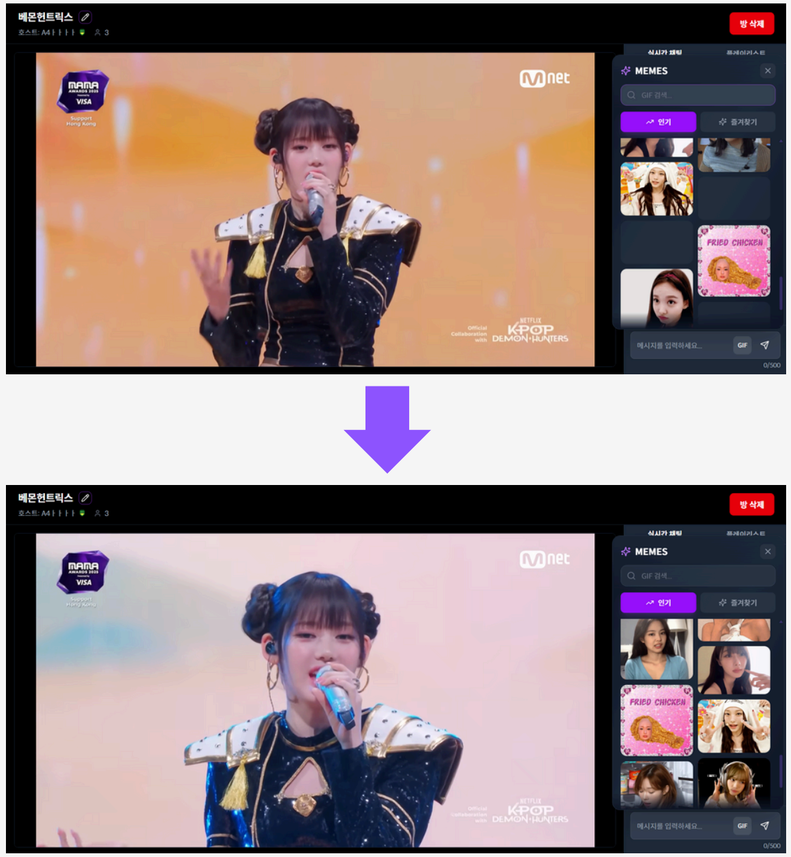


MemeDuck & DuckOn

Project 02

트러블슈팅 & 해결

- GIF 모달 렌더링 최적화 및 초기 지연
  - 실시간 채팅 화면에서 대부분의 사용자가 GIF 버튼을 자주 클릭해 밈 이미지를 전송
  - 모달 내 썸네일들이 모두 보이기까지 약 1.5초 ~ 2초의 딜레이 존재
  - 초기 API 호출과 GIF 이미지 디코딩 비용이 주요 원인
  - 해결 방안
    - Skeleton UI 도입으로 체감 지연 제거 → 사용자 체감 80% 이상 감소
    - 이미지 로딩 지연 최소화를 위한 lazy + async decoding 적용
    - 불필요한 렌더링 방지를 위한 조건 분기 최적화
    - Mansonry 형태로 GIF 분할 렌더링으로 컬럼별로 자연스럽게 분산



MemeDuck & DuckOn

Project 02

트러블슈팅 & 해결

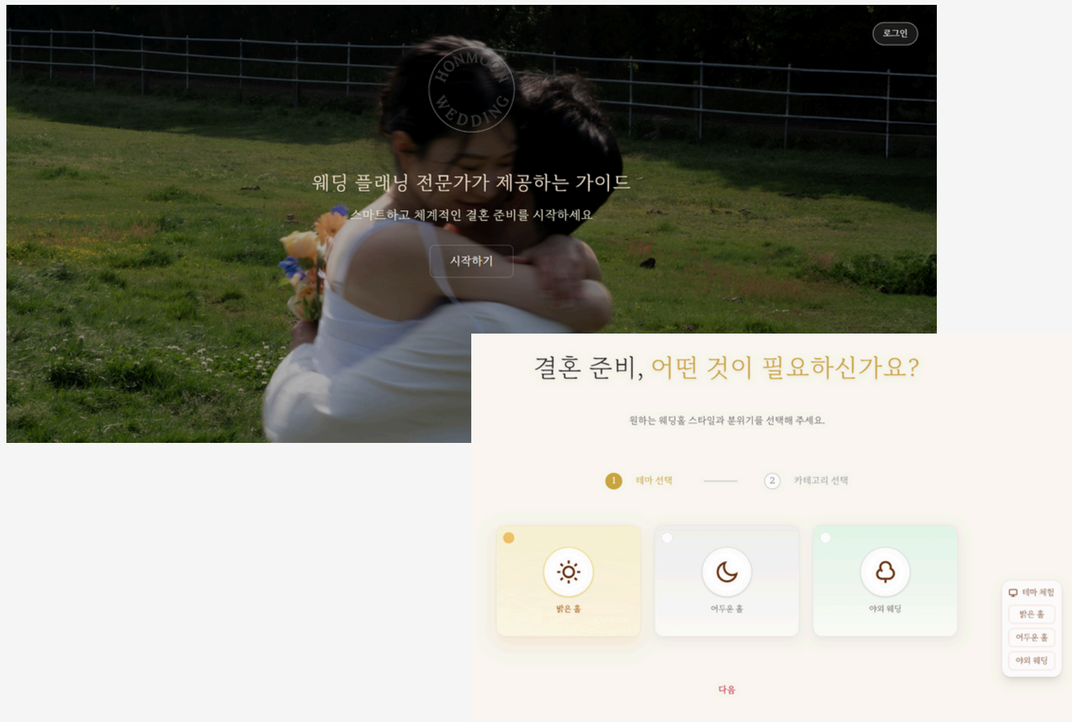
- GIF 검색 성능 최적화 및 검색 결과 정확도
  - 검색 속도, 정확도, UI 안정성이 중요
  - 짧은 검색어 입력에도 API가 과도하게 호출
  - 탭 전환 및 무한 스크롤에서 데이터 불일치 문제 발생
  - 즐겨찾기 상태 동기화 오류
  - 해결 방안
    - 불필요한 중복 검색 요청 제거를 통한 검색 성능 최적화
    - 의미 있는 검색어만 API 요청을 보내도록 최적화 → 250ms 디바운스 적용
    - 탭 전환 시 명확한 초기화를 통해 한 번에 한 요청만 수행되도록 로직 도입
    - 즐겨찾기 ID를 Set으로 관리하여 로드된 모든 데이터와 UI 즐겨찾기 표시 일치
    - 비동기 로그 처리를 통해 검색 속도 및 안정성 유지





Project 03

진행 프로젝트 소개



HonMoon(혼문)

프로젝트

HonMoon(혼문)은 예비 신혼부부가 결혼 준비 과정에서 겪는 정보 부족, 과도한 검색 및 비용 불투명성 문제를 해결하기 위한 '결혼 준비 정보 통합 플랫폼' 입니다.

개요

- ✔ 기간: 2025.08 ~ 2025.09
- ✔ 역할: 프론트엔드
- ✔ 팀 구성: 6인 팀(백엔드 3명, 프론트엔드 3명)
- ✔ 목표: 예비 신혼부부들이 쉽게 웨딩 준비를 할 수 있는 플래너

핵심 기능

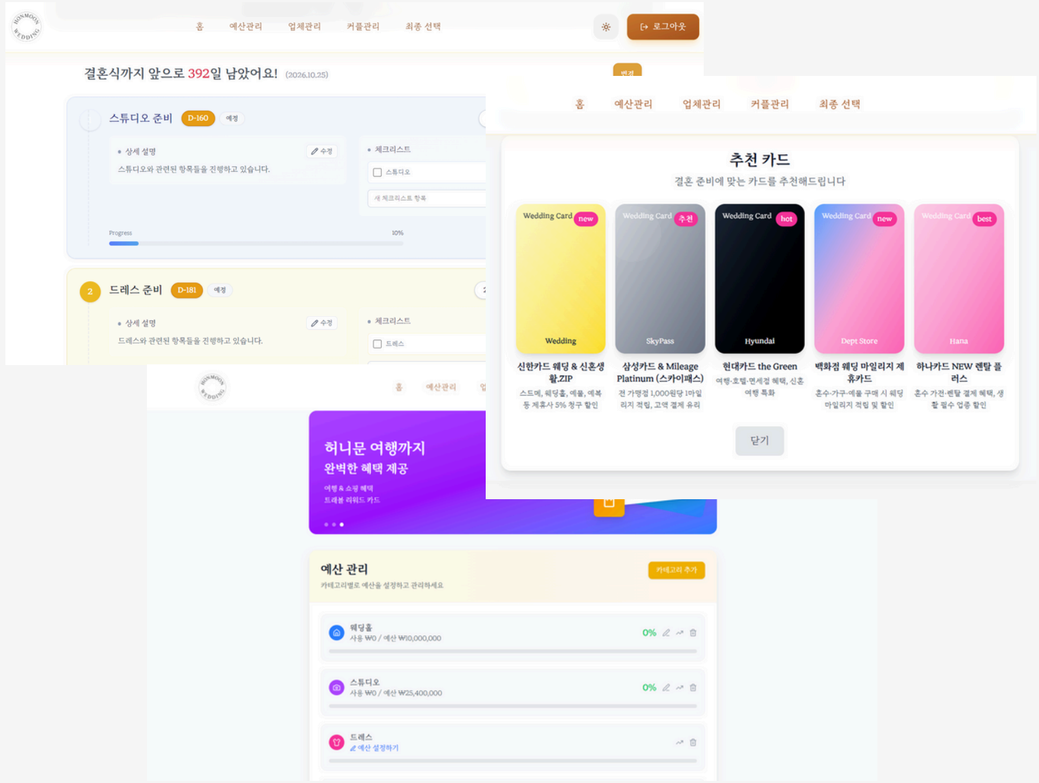
- ✔ 예비 신혼부부별 맞춤형 체크 리스트를 통한 카테고리 자동 생성
- ✔ 카테고리별 예산 현황 시각화
- ✔ 체계적인 일정 관리
- ✔ 업체 관리 및 최종 선택
- ✔ 예비 신혼부부의 소비 패턴에 따른 카드 추천

HonMoon(혼문)

Project 03

담당 역할

- 체크리스트 페이지 UI 개발
  - 예비 신혼부부가 결혼 준비 항목을 한 눈에 관리할 수 있도록 설계
  - 단계별 그룹화 및 진행률 표시
- 초대 코드 공유 흐름 설계 및 구현
  - 예비 부부 간 공동 체크리스트 공유를 위한 초대 코드 생성 및 입력 플로우 구축
  - 초대 코드 입력 → 검증 → 권한 연결까지의 전 과정을 설계
- 업체 관리 기능 및 UI 개발
  - 즐겨찾기, 비교, 가격/옵션/위치 기반 필터링 등 이요자 중심 재구성
- 최종 선택 UI 개발
  - 예비 신혼부부가 여러 업체 중 최종 선택을 확정할 수 있는 UI 제작
- 카드 등록 및 추천 카드 페이지 UI 개발
  - 예비 신혼부부의 소비 성향에 맞춘 카드 추천

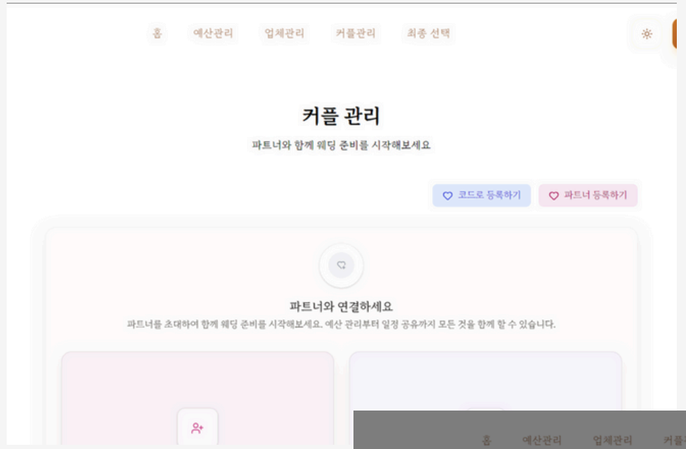


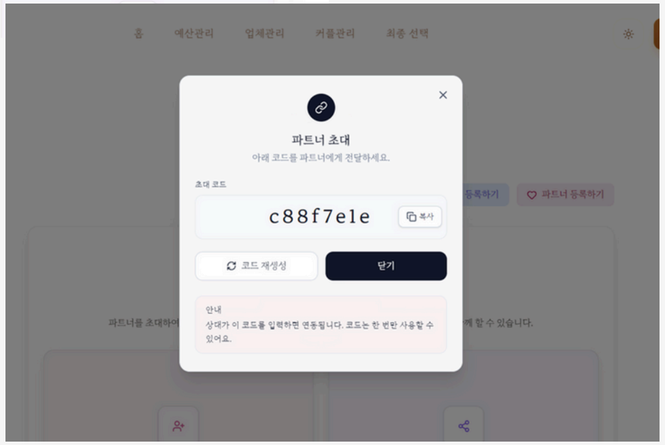
HonMoon(혼문)

Project 03

트러블슈팅 & 해결

- 커플 체크리스트 상태 불일치 문제
  - 예비 신혼부부가 서로 다른 체크리스트 화면을 봐서 불편함 발생
  - 한쪽에서 리스트를 체크해도 상대 화면에 즉시 반영되지 않음
  - 초기 설계가 사용자 기준으로 체크리스트를 분리 저장하고 커플 기능의 부재
  - 해결 방안
    - 초대 코드 기반 커플 연결 구조 구현
      - 한 사람이 초대 코드 생성 → 상대방이 입력해 수락
      - 두 사용자를 동일한 coupleId로 묶어 공동 데이터 구조 확립
    - 체크리스트 저장 구조를 개인 단위에서 커플 단위로 변경
      - coupleId → checklist[]로 개선하여 두 사람이 같은 데이터 소스를 공유
    - Zustand 기반 중앙 상태 관리로 통합
      - 컴포넌트별 개별 관리 방식 제거
      - 로그인 시 setCoupleId(coupleId)로 커플의 체크리스트 자동 로드
      - 페이지 이동 및 새로고침에도 일관된 UI 유지
    - 로그인 토큰 정책 맞춤 수정
      - 커플 연결 전에는 JWT 미발급 정책 도입
      - 토큰 없이도 개별 로그인은 가능하도록 설정





Thank you

제 포트폴리오를 봐주셔서 감사합니다.
새로운 도전을 두려워하지 않고, 더 나은 사용자 경험을 위해 끊임없이 고민하고 실행하겠습니다.